

DnDRom: world streaming, foliage and water

Research and implementation review | Unreal Engine 5.8.2 | Release 0.2.4 | September 7, 2026

Decision and scope

Improve the existing local procedural renderer without replacing its authoritative eroded terrain or requiring an external AI service. The design target remains 8 GB VRAM and 16 GB system RAM. This report covers loading, distant vegetation and object representation, water optics and wave motion. It distinguishes implemented changes from techniques that still require a separate implementation or benchmark.

The investigation combined original graphics literature, current Epic documentation, installed engine headers and shaders, code inspection, geometry tests and packaged application runs. The harbor fixture uses the existing terrain-first site at world coordinates (-3264, 4608), seed 719. The local map and streamed world retain that site's shared elevation datum and erosion fields.

The investigation identified both erosion and transport overhead. The final run spent about 6.8 seconds building its final resident world meshes in Unreal. Erosion and browser-to-native delivery must therefore be measured separately. Summed transfer durations include waiting behind queued transfers and are not additive wall-clock stages.

Evidence and engineering choices

Finch describes separate geometric undulations and finer normal detail, with continuous wave functions and analytic derivatives. This offers a bounded alternative to an FFT ocean for a lake or sheltered harbor. It is an approximation to water motion, not a complete fluid simulation. [Effective Water Simulation from Physical Models](#).

Epic's Single Layer Water combines a surface response with a volume model, uses scene depth and color behind water, and supports scattering, absorption and refraction in a dedicated pass. Its Opaque blend mode should not be confused with an ordinary opaque surface shader. [Single Layer Water Shading Model](#).

Epic recommends instancing repeated mesh assets and profiling the ISM/HISM tradeoff. HISM is useful for large static populations, while enabling a project-level Nanite setting does not build Nanite data for this application's imported meshes. This release keeps conventional runtime meshes and shares their buffers. [Instanced Static Mesh Component](#). The last statement was additionally verified in DnDRom's BuildFromMeshDescriptions path.

Epic's impostor baker uses multiple captured directions, blending nearby views; an upper-hemisphere atlas is suitable for trees viewed from above. Its editor baking workflow is not automatically available for arbitrary meshes generated inside an installed game. DnDRom's two crossed albedo cards lacked the view coverage and lighting information of that method. [Impostor Baker Plugin](#).

Geometry clipmaps use nested regular grids, reuse geometry and update bounded regions as the viewer moves. Transition bands reconcile levels. Their historical performance figures are not a benchmark for DnDRom. These principles support a future GPU heightfield renderer, but this release retains the existing quadtree and its tested parent/child coverage rules. [Terrain Rendering Using GPU-Based Geometry Clipmaps](#).

World Partition HLOD represents unloaded distant actors with simplified mesh/material proxies. Its documented build workflow is oriented around authored world content. For DnDRom, persistent settlement identities and runtime proxy generation must precede adopting that workflow for newly generated towns. HLOD does not itself invent a coherent distant settlement or road network. [World Partition - Hierarchical Level of Detail](#).

Epic's PSO precaching compiles pipeline states asynchronously and can delay component drawing until they are ready. Its documentation warns that broad compilation parallelism can consume substantial memory and compete with gameplay. This release limits the PSO compilation pool to two threads. A clean driver-cache benchmark remains separate from a cold terrain-cache test. [PSO Precaching](#).

McDonald's method couples transported sediment, time-averaged flow and momentum with local erosion and settling. Those evolving fields explain why generating the first region is more expensive than evaluating noise. DnDRom retains its existing adaptation and its solver/cache revision; this release does not substitute a cheaper un-eroded landscape. [Procedural Hydrology: Improvements and Meandering Rivers](#).

Implemented loading changes

Water mesh export now indexes shared vertices. A completely wet 256 m tile retains 8,192 triangles while reducing vertices from 24,576 to 4,225. Its geometry JSON shrinks from 988,884 bytes to 263,248; gzip plus base64 is 61,084 characters. This controlled geometry measurement is independent of GPU load. It is not a claim that the entire application loads sixteen times faster.

Scene and tile transfers negotiate gzip with the native host. Older hosts can decline and receive the original JSON. Small payloads use one bounded inline call after negotiation; larger payloads retain acknowledged chunks. The existing 512 KiB request boundary remains, and native decompression rejects invalid encodings and output sizes above 128 MiB. The geometry still passes the full native import validator.

The world update combines visibility changes and camera/readiness diagnostics into one native round trip on supporting hosts. Tile generation can refill its bounded queue without waiting for a separate visibility acknowledgement. Older native hosts retain the two-call path. A camera-transition test checks that the delayed visibility submission retains already displayed terrain coverage.

Repeated nearby tree prototypes are generated once per worker. Later tiles reference definitions already validated by the native host instead of resending their numeric arrays. That definition cache has a 16 MiB serialized-data budget and a 64-entry limit; actual object memory is larger than serialized size. Native tree and grass mesh buffers are cached by definition content, including geometry, material, collision flags and LODs. Weak cache entries allow the buffers to be reclaimed when their live components disappear. The cache is bounded to 256 entries. Tile imports share one nominal 4 ms budget per frame; one mesh build remains non-preemptible and can exceed that budget.

The disk cache now reads field keys before retrieving data, loads up to 96 fields nearest the scene, and retains up to 192 persisted fields. This avoids loading every distant field merely to discard most of them and protects the active scene from distant-cache churn. Failed disk persistence still falls back to deterministic in-memory generation. Native and browser tile limits now agree on three temporary replacement slots above the 512-tile steady-state budget.

A separate attempt to cache erosion-neighbor stencils preserved all six output arrays byte for byte but did not provide a meaningful speedup. It was removed. The useful changes came from measuring the actual generation/import pipeline, reducing duplicate data and reducing bridge round trips.

Implemented visual changes

Water uses a cooked Single Layer Water material with separate animated world-space wave normals and two gentle geometric waves. Geometry displacement fades continuously with camera distance, rather than switching by tile ID. A 16 cm material displacement bound encloses the maximum 12.5 cm offset. Small pixel waves are filtered when they become smaller than the screen sampling footprint.

The material reads shoreline depth from mesh vertex data and reduces wave strength near dry edges. Absorption and scattering create depth-dependent appearance; refraction uses a bounded pixel-normal offset to avoid displacing the entire lakebed out of view. The installed UE 5.8 SingleLayerWaterMaterialOutput header specifies

reciprocal centimeters for its raw coefficients, so values expressed per meter must be divided by 100. These units were checked against installed source rather than inferred from similarly named plugin presets.

Shallow foam is a moving procedural breakup near the encoded shore depth. It is not a hydrodynamic foam simulation. The shader currently uses a fixed gentle wave spectrum; weather-driven wind, directional river advection, breaking surf, underwater transitions and physically simulated caustics remain future work.

Middle-distance trees now use four side projections and an overhead projection of the actual procedural branch graph. Leaf coverage is denser than the original sparse card texture. A dedicated foliage shader selects useful planes by viewing angle and corrects two-sided lighting so back faces do not become black crossed silhouettes. Each tree uses ten triangles and shared textures. This is a low-cost multi-plane approximation, not a parallax-correct octahedral impostor. Solid 3D crown proxies were tested but rejected in the middle distance because their rounded lobes were too obvious.

Far crowns keep their physical dimensions as LOD changes. Each crown footprint is checked for water, slope and treeline compatibility. They use a foliage material instead of a grass-ground texture. At distances where individual trees become subpixel, woodland coverage blends into the terrain material, gated by rock and snow weights. The prior LOD-sized opaque fans are no longer used.

Material generation is versioned, and the packaging command now rejects material compilation failures even when Unreal otherwise reports a successful cook. Authored terrain PBR maps and stochastic texturing remain active.

Measured validation and limits

The final empty-terrain-cache harbor run reached complete nearby selected coverage in 211.4 seconds. The same import with a persisted erosion cache in an otherwise fresh browser profile took 218.5 seconds. These are single offscreen packaged runs on a machine with an RTX 3080 (10 GB VRAM) and 32 GB RAM. Background load was uncontrolled: another game was active earlier in testing and had exited by the final cold run's completion. These results are not a controlled speedup comparison or a verified 16 GB minimum-spec result. The earlier instrumented candidate took 448.6 seconds under shared load and still used tree cards and the uncompressed transfer path; its timing is contextual evidence rather than a statistically controlled baseline. The warm run built zero new erosion fields, but delivery and import scheduling still caused a substantial wait.

The final camera test sampled 90 transitions and lost 0 previously covered probes. Its final resident tile count was 204, with 153 reused prototype builds and no missing materials. The application smoke verified 26 UI/native behaviors, including compressed-size and corrupt-input rejection. Focused geometry, transfer and visibility tests passed; the wider atlas tests were also run.

Across the final cold review, the bridge recorded 111,961,697 source JSON characters and 33,331,529 transmitted characters in 240 data calls. These counters include the scene import and subsequent camera streaming, so they are not a per-tile average or a network download measurement. All communication remains inside the local application.

Screenshots were reviewed at the initial camera and a farther camera angle, with a second stationary capture to inspect water animation. Coverage probes establish residency continuity, not the absence of every visible LOD transition. The distant forest approximation remains stylized. First-time hydraulic generation remains a noticeable wait, and this release is not an instant-world generator.

Next measured steps

The next performance milestone is a controlled idle-machine cold/warm comparison with CPU, GPU and memory traces, followed by a reference-conformant native or WASM erosion backend if simulation remains dominant. A solver migration needs a deliberate version policy because tiny floating-point changes can alter evolving terrain. It must not silently change persisted campaign landscapes.

For richer forests, benchmark baked upper-hemisphere albedo/normal/depth atlases against the current multi-plane impostors and far crowns at equal screen coverage and vegetation density. Compare overdraw, triangle processing, temporal stability, memory and treeline transitions. Do not choose the lowest triangle count without measuring alpha cost and image quality.

For distant castles, towns and roads, persist one semantic world graph with stable IDs and build simplified representations from the same settlement layouts used at encounter scale. A destination visible on a mountain must refer to the actual destination. That content pipeline is still distinct from this release's terrain, foliage and water work.

Source provenance

S1. Mark Finch / Cyan Worlds. 2004. [Effective Water Simulation from Physical Models](#). Primary technical source.

S2. Epic Games. UE 5.8; accessed 2026-09-07. [Single Layer Water Shading Model](#). Primary technical source.

S3. Epic Games. UE 5.8; accessed 2026-09-07. [Instanced Static Mesh Component](#). Primary technical source.

S4. Epic Games. UE 5.8; accessed 2026-09-07. [Impostor Baker Plugin](#). Primary technical source.

S5. Arul Asirvatham and Hugues Hoppe. 2005. [Terrain Rendering Using GPU-Based Geometry Clipmaps](#). Primary technical source.

S6. Epic Games. UE 5.8; accessed 2026-09-07. [World Partition - Hierarchical Level of Detail](#). Primary technical source.

S7. Epic Games. UE 5.8; accessed 2026-09-07. [PSO Precaching](#). Primary technical source.

S8. Nicholas McDonald. 2023-12-12. [Procedural Hydrology: Improvements and Meandering Rivers](#). Primary technical source.

Local evidence: scripts/unreal/water_surface.hlsli; installed UE 5.8 material header and shader source; native-polish-release cold/warm reports; native-polish-ui-release report; water-transfer-benchmark.json; hydrology-024-benchmark.json.